Lab 7: Motor Control of the Pololu 3pi+ Robot using Pulse-Width Modulation

10/28/2026

Daniel Xu
ECE 3218L

1 Overview

The Pololu 3pi+ 32U4 robot has two DC motors, each driven by a DRV8838 driver with a direction pin (PH) and an enable pin (EN). In this lab, the motors were driven at full speed by switching EN on and off, then at quarter speed with Timer1 phase-correct PWM. All five Timer1 prescalers were tested to find the best PWM frequency.

2 Procedure

- **Setup** Table 1 lists the Port B pins. The motor pins start low, which brakes both motors. Button A uses the internal pull-up, and the program waits for a press of A before moving.

Table 1: Port B pins used in this lab.

Pin	Function	Mode
PB1	Right motor direction (PH)	Output
PB2	Left motor direction (PH)	Output
PB3	Button A	Input, pull-up
PB5	Right motor enable (EN)	Output, Timer1 OC1A
PB6	Left motor enable (EN)	Output, Timer1 OC1B

- **Full-speed tests** With TCCR1A cleared, the motor pins are written directly (Table 2). By writing EN high and then PH = 0 for forward or PH = 1 for reverse, the robot moves in alternating directions for 1 s each time. Between each move, EN is pulled low, which brakes the robot for 3 s.

Table 2: Port B motor bits for each move (bits 6, 5, 2, 1 are left EN, right EN, left PH, right PH).

Move	Binary
Brake	00000000
Left forward	01000000
Left reverse	01000100
Right forward	00100000
Right reverse	00100010
Both forward	01100000
Both reverse	01100110
Spin CCW (left reverse, right forward)	01100100
Spin CW (left forward, right reverse)	01100010

- **Quarter-speed tests** Timer1 runs in 8-bit phase-correct PWM (Mode 1, TOP = 255), non-inverting on OC1A and OC1B (TCCR1A = 0xA1). CS12:10 select the prescaler N, and since the counter counts up to TOP and back down,

$$f_{\text{PWM}} = \frac{16\text{ MHz}}{2 \cdot N \cdot \text{TOP}} = \frac{16\text{ MHz}}{2 \cdot N \cdot 255}. \tag{1}$$

  Quarter speed is OCR1x = 64 (25% duty). Each move runs for 2 s, then brakes for 3 s.

- **Prescaler experiment** After each quarter-speed run, pressing A switches to the next prescaler and repeats the test.

3 Results

3.1 Motor Tests

Every full-speed and quarter-speed move went in the expected direction.

3.2 OLED

The test program showed ECE3218 as ECE321, because the provided `OLED_writeText()` always draws exactly 6 characters. It was changed to stop at the end of the string and blank the rest of the line, which fits up to 16 characters.

3.3 Prescaler Experiment

Table 3: Quarter-speed behavior for each Timer1 prescaler.

N	CS12:10	f_{PWM}	Period	Observed behavior
1	001	31.4 kHz	31.9 μs	Smooth and quiet.
8	010	3.92 kHz	255 μs	Smoother than $N = 1$, with a slight high-pitched whine.
64	011	490 Hz	2.04 ms	Smooth, with slight rattling.
256	100	123 Hz	8.16 ms	Same as $N = 64$.
1024	101	30.6 Hz	32.6 ms	Rough and rattling.

4 Post-Lab

Comment on any unexpected behaviors exhibited by the robot during the experiments.
$N = 8$ felt smoother than $N = 1$, which was expected to be the smoothest (explained below). Otherwise, the robot behaved as expected. The only difference was that $N = 8$ produced a slight whine in the motor.

Explain the difference in behavior between non-inverting and inverting control methods.
Non-inverting output is high while TCNT1 < OCR1x (duty = OCR1x/255), and inverting output is high while TCNT1 > OCR1x (duty = 1 − OCR1x/255). Since EN low brakes, the same OCR value gives opposite speeds:

- **Non-inverting:** OCR1x = 0 brakes, 64 is quarter speed, and 255 is full speed.
- **Inverting:** OCR1x = 255 brakes, 191 is quarter speed, and 0 is full speed.

For the same duty cycle, the motor behaves identically in both modes. Non-inverting makes more sense intuitively, since a larger OCR means a faster motor and 0 means stopped.

Describe the results of part 3 from the in-lab portion for each of the prescaler values.

- $N = 1$ **(31.4 kHz).** Above the 20 kHz limit of hearing, so silent. Motor feels and sounds smooth.
- $N = 8$ **(3.92 kHz).** This is a normal frequency that humans can hear, which caused a whine.
- $N = 64$ **(490 Hz) and** $N = 256$ **(123 Hz).** The current rises and falls each cycle, giving a torque pulse heard as a slight rattle.
- $N = 1024$ **(30.6 Hz).** The 32.6 ms period lets the motor speed up and slow down each cycle, so the robot moved roughly.

Justify your choice of the best value to use.
$N = 1$, the only setting that is both silent and free of torque ripple. $N = 8$'s whine is a real drawback that puts it below $N = 1$. All other values produced weird torque rippling and sounds.

5 Discussion/Conclusion

The 3pi+ motors were driven at full speed with on/off outputs and at quarter speed with Timer1 phase-correct PWM, and every move went in the expected direction. $N = 1$ (31.4 kHz) was the best prescaler: silent and smooth, while $N = 8$ whined, $N = 64$ and 256 rattled slightly, and $N = 1024$ was rough.

A Full Code

Code 1: Test sequence and prescaler experiment (main.c).

```
//   1. Setup: brake motors, wait for button A.
//   2. Full-speed test, one motor at a time.
//   3. Full-speed test, both motors together.
//   4. Quarter-speed tests using Timer1 phase-correct PWM.

#ifndef F_CPU
#define F_CPU 16000000UL
#endif

#include <avr/io.h>
#include <util/delay.h>
#include <stdio.h>
#include "motors.h"
#include "button.h"

// 1 = show each step on the OLED (needs font.c, OLED.c, OLED.h in this
// folder). Set to 0 to build without them.
#define USE_OLED 1

// Prescaler for the first run of part 4 (see pwm_prescale_t in motors.h)
#define START_PRESCALE  PRESCALE_64

// 0 = non-inverting PWM, 1 = inverting PWM (for the post-lab question)
#define PWM_INVERTING   0

#define RUN_FULL_MS   1000   // full-speed moves: 1 s
#define RUN_PWM_MS    2000   // quarter-speed moves: 2 s
#define PAUSE_MS      3000   // "brake and wait a few seconds"

#if USE_OLED
#include "OLED.h"

static char text[50];

static void show(const char *msg)
{
    snprintf(text, sizeof text, "%s", msg);
    OLED_writeText(text);
}

static void oled_setup(void)
{
    OLED_init();
    OLED_reset();
    OLED_clearDisplayRam();
    OLED_configureDefault();
    show("Lab 7");
}
#else
static void show(const char *msg) { (void)msg; }   // does nothing
static void oled_setup(void) { }
#endif

// _delay_ms() wants a constant, so loop 1 ms at a time
```

```
55 static void delay_ms(uint16_t ms)
56 {
57     while (ms--)
58         _delay_ms(1);
59 }
60
61 // Brake both motors and wait
62 static void brake_and_wait(void)
63 {
64     motors_brake();
65     show("Brake");
66     delay_ms(PAUSE_MS);
67 }
68
69 // ---- Part 2: full speed, one motor at a time ----------------------------
70 static void part2_single_motors(void)
71 {
72     show("L fwd");
73     motors_set_full(MOTOR_FORWARD, MOTOR_BRAKE);
74     delay_ms(RUN_FULL_MS);
75     brake_and_wait();
76
77     show("L rev");
78     motors_set_full(MOTOR_REVERSE, MOTOR_BRAKE);
79     delay_ms(RUN_FULL_MS);
80     brake_and_wait();
81
82     show("R fwd");
83     motors_set_full(MOTOR_BRAKE, MOTOR_FORWARD);
84     delay_ms(RUN_FULL_MS);
85     brake_and_wait();
86
87     show("R rev");
88     motors_set_full(MOTOR_BRAKE, MOTOR_REVERSE);
89     delay_ms(RUN_FULL_MS);
90     brake_and_wait();
91 }
92
93 // Part 3: full speed, both motors
94 static void part3_both_motors(void)
95 {
96     show("Fwd");
97     motors_set_full(MOTOR_FORWARD, MOTOR_FORWARD);
98     delay_ms(RUN_FULL_MS);
99     brake_and_wait();
100
101     show("Rev");
102     motors_set_full(MOTOR_REVERSE, MOTOR_REVERSE);
103     delay_ms(RUN_FULL_MS);
104     brake_and_wait();
105
106     // Spinning CCW (seen from above): left wheel back, right wheel forward
107     show("CCW");
108     motors_set_full(MOTOR_REVERSE, MOTOR_FORWARD);
109     delay_ms(RUN_FULL_MS);
110     brake_and_wait();
111
112     // Spinning CW: left wheel forward, right wheel back
113     show("CW");
```

```
    motors_set_full(MOTOR_FORWARD, MOTOR_REVERSE);
    delay_ms(RUN_FULL_MS);
    brake_and_wait();
}

// Part 4: quarter speed with PWM
static void part4_quarter_speed(void)
{
    const int16_t q = MOTOR_SPEED_QUARTER;

    show("1/4fwd");
    motors_set_speed(q, q);
    delay_ms(RUN_PWM_MS);
    brake_and_wait();

    show("1/4rev");
    motors_set_speed(-q, -q);
    delay_ms(RUN_PWM_MS);
    brake_and_wait();

    show("1/4CCW");
    motors_set_speed(-q, q);
    delay_ms(RUN_PWM_MS);
    brake_and_wait();

    show("1/4 CW");
    motors_set_speed(q, -q);
    delay_ms(RUN_PWM_MS);
    brake_and_wait();
}

// Label for the OLED, e.g. "N=64"
static const char *prescale_label(pwm_prescale_t p)
{
    switch (p) {
        case PRESCALE_1:    return "N=1";
        case PRESCALE_8:    return "N=8";
        case PRESCALE_64:   return "N=64";
        case PRESCALE_256:  return "N=256";
        case PRESCALE_1024: return "N=1024";
        default:            return "N=?";
    }
}

int main(void)
{
    // Part 1: setup
    motors_init();          // motor pins as outputs, both braked
    button_a_init();        // PB3 input with pull-up
    oled_setup();

    show("Push A");
    button_a_wait();        // don't move until button A is pressed
    delay_ms(500);          // time to get your hand clear

    part2_single_motors();
    part3_both_motors();

    // Switch from digital mode to Timer1 PWM
```

```
    pwm_prescale_t prescale = START_PRESCALE;
    motors_pwm_init(prescale, PWM_INVERTING);
    show(prescale_label(prescale));
    delay_ms(1000);
    part4_quarter_speed();

    // Prescaler experiment: A = next prescaler, repeat part 4
    for (;;) {
        show("A=next");
        button_a_wait();

        prescale = (prescale >= PRESCALE_1024) ? PRESCALE_1 : prescale + 1;
        motors_pwm_set_prescale(prescale);

        show(prescale_label(prescale));
        delay_ms(1000);
        part4_quarter_speed();
    }
}
```

Code 2: Motor driver interface (motors.h).

```
#ifndef MOTORS_H
#define MOTORS_H

#include <stdint.h>

// Direction / state
typedef enum {
    MOTOR_BRAKE = 0,
    MOTOR_FORWARD,
    MOTOR_REVERSE
} motor_state_t;

typedef enum {
    PRESCALE_1    = 1,   // ~31.4 kHz
    PRESCALE_8    = 2,   // ~3.92 kHz
    PRESCALE_64   = 3,   // ~490 Hz
    PRESCALE_256  = 4,   // ~123 Hz
    PRESCALE_1024 = 5    // ~30.6 Hz
} pwm_prescale_t;


#define MOTOR_SPEED_MAX     255
#define MOTOR_SPEED_QUARTER  64

void motors_init(void);

void motors_set_full(motor_state_t left, motor_state_t right);

void motors_pwm_init(pwm_prescale_t prescale, uint8_t inverting);

void motors_pwm_set_prescale(pwm_prescale_t prescale);

void motors_set_speed(int16_t left, int16_t right);

void motors_brake(void);

```

```
#endif // MOTORS_H
```

Code 3: Motor driver: full-speed and PWM modes (motors.c).

```
// See motors.h for the pin map and an overview.

#include <avr/io.h>
#include "motors.h"

// Pin definitions (all on Port B)
#define R_DIR  PB1   // right motor direction
#define L_DIR  PB2   // left  motor direction
#define R_PWM  PB5   // right motor speed, OC1A
#define L_PWM  PB6   // left  motor speed, OC1B

// Direction pin level for FORWARD. Pololu's library drives DIR low for
// forward, so 0 here. If a motor turns the wrong way on your robot,
// change that motor's value to 1.
#define L_FWD_LEVEL 0
#define R_FWD_LEVEL 0

static uint8_t pwm_inverting = 0;   // remembered from motors_pwm_init()

// Set a direction pin for forward (reverse = 0) or reverse (reverse = 1).
static void set_dir(uint8_t pin, uint8_t fwd_level, uint8_t reverse)
{
    uint8_t level = fwd_level ^ reverse;   // flip the level for reverse
    if (level)
        PORTB |=  (1 << pin);
    else
        PORTB &= ~(1 << pin);
}
static uint8_t duty_to_ocr(uint8_t duty)
{
    return pwm_inverting ? (uint8_t)(255 - duty) : duty;
}

void motors_init(void)
{
    // Disconnect Timer1 from the pins so PORTB controls them directly
    TCCR1A = 0;
    TCCR1B = 0;

    // All four motor pins low (= brake), then make them outputs
    PORTB &= ~((1 << R_DIR) | (1 << L_DIR) | (1 << R_PWM) | (1 << L_PWM));
    DDRB  |=  ((1 << R_DIR) | (1 << L_DIR) | (1 << R_PWM) | (1 << L_PWM));
}

// One motor in digital mode: speed pin HIGH = full speed, LOW = brake.
static void set_full_one(uint8_t dir_pin, uint8_t pwm_pin,
                         uint8_t fwd_level, motor_state_t state)
{
    if (state == MOTOR_BRAKE) {
        PORTB &= ~(1 << pwm_pin);
    } else {
        set_dir(dir_pin, fwd_level, state == MOTOR_REVERSE);
        PORTB |= (1 << pwm_pin);
    }
```

```
}

void motors_set_full(motor_state_t left, motor_state_t right)
{
    set_full_one(L_DIR, L_PWM, L_FWD_LEVEL, left);
    set_full_one(R_DIR, R_PWM, R_FWD_LEVEL, right);
}

void motors_pwm_init(pwm_prescale_t prescale, uint8_t inverting)
{
    pwm_inverting = inverting ? 1 : 0;

    // Start with both motors braked
    OCR1A = duty_to_ocr(0);
    OCR1B = duty_to_ocr(0);

    // TCCR1A:
    //   COM1A1:0 and COM1B1:0 = 10 non-inverting, 11 inverting
    //   WGM11:10 = 01  -> with WGM13:12 = 00 this is Mode 1,
    TCCR1A = (1 << COM1A1) | (1 << COM1B1) | (1 << WGM10);
    if (pwm_inverting)
        TCCR1A |= (1 << COM1A0) | (1 << COM1B0);

    // TCCR1B: WGM13:12 = 00, CS12:10 = prescaler (starts the timer)
    TCCR1B = (uint8_t)prescale & 0x07;
}

void motors_pwm_set_prescale(pwm_prescale_t prescale)
{
    TCCR1B = (TCCR1B & ~0x07) | ((uint8_t)prescale & 0x07);
}

// Clamp a signed speed to -255..255
static int16_t clamp_speed(int16_t s)
{
    if (s >  MOTOR_SPEED_MAX) return  MOTOR_SPEED_MAX;
    if (s < -MOTOR_SPEED_MAX) return -MOTOR_SPEED_MAX;
    return s;
}

void motors_set_speed(int16_t left, int16_t right)
{
    left  = clamp_speed(left);
    right = clamp_speed(right);

    set_dir(L_DIR, L_FWD_LEVEL, left  < 0);
    set_dir(R_DIR, R_FWD_LEVEL, right < 0);

    OCR1B = duty_to_ocr((uint8_t)(left  < 0 ? -left  : left));   // left  = OC1B
    OCR1A = duty_to_ocr((uint8_t)(right < 0 ? -right : right));  // right = OC1A
}

void motors_brake(void)
{
    if (TCCR1A & ((1 << COM1A1) | (1 << COM1B1))) {
        // PWM mode: 0% duty holds both speed pins low
        OCR1A = duty_to_ocr(0);
        OCR1B = duty_to_ocr(0);
    } else {
```

```
        // Digital mode
        PORTB &= ~((1 << R_PWM) | (1 << L_PWM));
    }
}
```

Code 4: Button A interface (button.h).

```
// Button A is on PB3. It connects the pin to ground when pressed, so we
// turn on the ATmega32U4's internal pull-up: released = HIGH, pressed = LOW.

#ifndef BUTTON_H
#define BUTTON_H

#include <stdint.h>

// Make PB3 an input with its pull-up enabled.
void button_a_init(void);

// 1 if button A is held down right now, 0 if not.
uint8_t button_a_pressed(void);
void button_a_wait(void);

#endif // BUTTON_H
```

Code 5: Button A input with debouncing (button.c).

```
#ifndef F_CPU
#define F_CPU 16000000UL
#endif

#include <avr/io.h>
#include <util/delay.h>
#include "button.h"

#define BTN_A  PB3

void button_a_init(void)
{
    DDRB  &= ~(1 << BTN_A);   // input
    PORTB |=  (1 << BTN_A);   // enable internal pull-up
    _delay_us(10);            // let the pin settle high
}

uint8_t button_a_pressed(void)
{
    return (PINB & (1 << BTN_A)) == 0;   // LOW = pressed
}

void button_a_wait(void)
{
    do {
        while (!button_a_pressed())
            ;                    // loop indefinitely until pressed
        _delay_ms(20);           // debounce
    } while (!button_a_pressed());

    // Wait for release, again debounced
    do {
```

```
        while (button_a_pressed())
            ;
        _delay_ms(20);
    } while (button_a_pressed());
}
```

Code 6: Course-provided OLED driver header, with OLED_TEXT_MAX added (OLED.h).

```
/*
 * OLED.h
 *
 * Created: 9/19/2024 7:12:26 PM
 *  Author: sternbal
 */


#ifndef OLED_H_
#define OLED_H_

#define OLEDCLK PD3 // D1
#define OLEDRES PD2 // D0
#define OLEDDC PB0 // D17
#define OLEDMOSI PD5
// Characters per line of OLED_writeText(): text starts at column 16 and
// each character is 7 pixels wide (5 glyph + 2 gap), so 16 fit in 128.
#define OLED_TEXT_MAX 16
void OLED_clk_low(void);
void OLED_clk_high(void);
void OLED_mos_pin(uint8_t);
void OLED_write(uint8_t);
void OLED_commandMode(void);
void OLED_dataMode(void);
void OLED_reset(void);
void OLED_clearDisplayRam(void);
void OLED_configureDefault(void);
void OLED_writeText(char*);
void OLED_init(void);
uint8_t getGlyphColumn(uint8_t, uint8_t);
#define SET_COL_ADDR_LOW 0x00
#define SET_COL_ADDR_HIGH 0x10
#define SET_CONTRAST 0x81
#define SET_SEG_REMAP 0xA0
#define SET_INV_DISP 0xA6
#define SET_DISPLAY_ON 0xAE
#define SET_PAGE_ADDR 0xB0
#define SET_COM_SCAN_DIR 0xC0

#endif /* OLED_H_ */
```

Code 7: Course-provided OLED driver, with OLED_writeText() modified (OLED.c).

```
/*
 * OLED.c
 *
 * Created: 9/19/2024 9:50:58 PM
 *  Author: sternbal
 */
#include <avr/io.h>
```

```
#include "OLED.h"
#define F_CPU 16000000L
#include <util/delay.h>
#include <avr/pgmspace.h>

extern const PROGMEM uint8_t pololuOledFont[][5];


void OLED_init(void)
{
    // set reset pin to output
    DDRD |= 1<<OLEDRES;

    // set clock pin to output and make it low
    DDRD |= 1<<OLEDCLK;
    PORTD &= ~(1<<OLEDCLK);

    // set MOSI pin to output
    DDRD |= 1<<OLEDMOSI;


    // set data/command pin to output
    DDRB |= 1<<OLEDDC;

}

void OLED_clk_low(void)
{
    PORTD &= ~(1<<OLEDCLK);

}
void OLED_clk_high(void)
{
    PORTD |= (1<<OLEDCLK);

}

void OLED_mos_pin(uint8_t val)
{
    if(val==0) PORTD &= ~(1<<OLEDMOSI);
    else PORTD |= 1<<OLEDMOSI;
}

void OLED_reset(void)
{
    PORTD &= ~(1<<OLEDRES);
    _delay_us(10);
    PORTD |= (1<<OLEDRES);
    _delay_us(10);
}


void OLED_clearDisplayRam(void)
{
    OLED_clk_low();
    OLED_commandMode();
    OLED_write(SET_COL_ADDR_LOW | 2);
    for (uint8_t page = 0;page < 8; page++)
    {
```

```
        OLED_commandMode();
        OLED_write(SET_PAGE_ADDR | page);
        OLED_write(SET_COL_ADDR_HIGH | 0);
        OLED_dataMode();
        for (uint8_t i = 0 ; i < 128; i++)
        {
            OLED_write(0);
        }
    }
}

void OLED_configureDefault(void)
{
    OLED_clk_low();
    OLED_commandMode();
    OLED_write(SET_SEG_REMAP | 1);
    OLED_write(SET_COM_SCAN_DIR | 8);
    OLED_write(SET_CONTRAST);
    OLED_write(0xFF);
    OLED_write(SET_DISPLAY_ON | 1);
}

void OLED_commandMode(void)
{
    PORTB &= ~(1<<OLEDDC);
}

void OLED_dataMode(void)
{
    PORTB |= (1<<OLEDDC);
}


void OLED_write(uint8_t d)
{
    OLED_clk_low();
    OLED_mos_pin(d >> 7 & 0x01);
    OLED_clk_high();

    OLED_clk_low();
    OLED_mos_pin(d >> 6 & 0x01);
    OLED_clk_high();

    OLED_clk_low();
    OLED_mos_pin(d >> 5 & 0x01);
    OLED_clk_high();

    OLED_clk_low();
    OLED_mos_pin(d >> 4 & 0x01);
    OLED_clk_high();

    OLED_clk_low();
    OLED_mos_pin(d >> 3 & 0x01);
    OLED_clk_high();

    OLED_clk_low();
    OLED_mos_pin(d >> 2 & 0x01);
    OLED_clk_high();

```

```
126     OLED_clk_low();
127     OLED_mos_pin(d >> 1 & 0x01);
128     OLED_clk_high();
129 
130     OLED_clk_low();
131     OLED_mos_pin(d >> 0 & 0x01);
132     OLED_clk_high();
133 }
134 
135 void OLED_writeText(char *text)
136 {
137     OLED_commandMode();
138     OLED_write(SET_PAGE_ADDR | 0);
139     OLED_write(SET_COL_ADDR_HIGH | 1);
140     OLED_write(SET_COL_ADDR_LOW | 0);
141     OLED_dataMode();
142     // Draw the string up to its '\0', then fill the rest of the row with
143     // spaces so any leftovers from a longer previous message are erased.
144     for (uint8_t i=0;i<OLED_TEXT_MAX;i++)
145     {
146         char glyph=' ';
147         if (*text) glyph= *text++;
148 
149         for (uint8_t pixelX=0;pixelX<5;pixelX++)
150         {
151             uint8_t column=getGlyphColumn(glyph,pixelX);
152             OLED_write(column);
153 //          OLED_write(column);
154         }
155         OLED_write(0);
156         OLED_write(0);
157     }
158 }
159 uint8_t getGlyphColumn(uint8_t glyph, uint8_t pixelX)
160 {
161     if(glyph>=0x20)
162     {
163         return pgm_read_byte(&pololuOledFont[glyph - 0x20][pixelX]);
164     }
165     return(0);
166 }
```

Code 8: OLED test from the handout (oled_test/main.c).

```
1  /*
2   * Step 1: Test out the OLED display
3   */
4  #ifndef F_CPU
5  #define F_CPU 16000000UL
6  #endif
7 
8  #include <avr/io.h>
9  #include <stdio.h>      /* for snprintf() */
10 #include "OLED.h"
11 
12 int main(void)
13 {
14     char text[50];              // define an array to hold text to be written to the display
```

```
15
16      OLED_init();                // set up the pins for input/output that connect to the display
17                                  // (the handout says OLED_Init, but the provided OLED.c names
        it OLED_init)
18      OLED_reset();               // toggle the display's reset pin
19      OLED_clearDisplayRam();     // display data is held in RAM. Zero everything to blank
        display
20      OLED_configureDefault();    // set mode, contrast, etc
21      snprintf(text, 8, "%s", "ECE3218");   // copy a string constant into our variable
22      OLED_writeText(text);       // display the string contents
23
24      while (1)
25          ;                       // stop here so the text stays on screen
26  }
```